

The allowlist swap

Changing a Daml model on a running participant works today, two different ways, using only APIs that already ship. This is the mechanism, the decision, the fix we build in DPM, and the one capability Canton has to expose.

Verified on Canton 3.5.12 • dpm-sdk 3.5.5 • OpenJDK 21 Reload ~12s, participant PID unchanged
Canton changes for the loop none

The problem. Editing one line of a template and testing it means either bumping the version and writing an upgrade — which fails for most edits made during development — or restarting the ledger and losing every party and contract.

The decision. Two approaches, split by whether the change is a valid upgrade. Compatible changes: bump the version, already fully supported. Incompatible changes: archive the contracts, then swap the package. Both work now.

The gap. Archiving leaves history that cannot be reclaimed on an in-memory participant. `repair.purge` does exactly this and is verified working on Postgres — it is refused on the sandbox.

The problem

Why the inner development loop is broken.

A package's identity is a hash of its code, so any edit produces a package Canton treats as unrelated to the previous one. Two routes exist for getting the new code onto a running participant, and both fail for ordinary development edits:

- **Same name and version** — rejected. Canton will not vet two packages sharing a name and version.
- **Bumped version** — rejected when the change is not a valid upgrade. Changing a field type, removing a field or changing a signatory all qualify, and all are routine while developing.

That leaves restarting. On `dpm sandbox` storage is in-memory, so a restart destroys every party and contract — and party IDs are not stable across restarts, so any connected application must reconnect and re-resolve the parties it works with.

What we considered

Three candidate designs, and the outcome of testing each.

A

Bump the version, and have Daml emit an event to DPM

Bump `1.0.0` → `1.0.1` carrying an incompatible change, delete the old version, and signal DPM so the sandbox knows the model moved.

The bump path is sound for compatible changes. For a type change Canton rejects the vetting, and no event can bridge it: existing contracts store the old shape, and a package expecting the new one cannot read those bytes. It is a data-shape impossibility, not a permissions check.

VIABLE

compatible changes

·

NOT VIABLE

incompatible changes

B

Archive the contracts, then replace the package

Archive the old contracts, then replace the package under the same name and version, keeping the participant running.

Verified end to end: no leftovers, participant PID unchanged, party IDs unchanged. Connected applications and PQS need no special handling — archiving uses ordinary Daml authority, so they receive real archive events and converge on their own.

CHOSEN

this is the approach we are proposing

C

Delete the archived contracts

Folded into B. Archiving clears the active set; the history records remain in the event log. On in-memory storage that grows until the participant restarts.

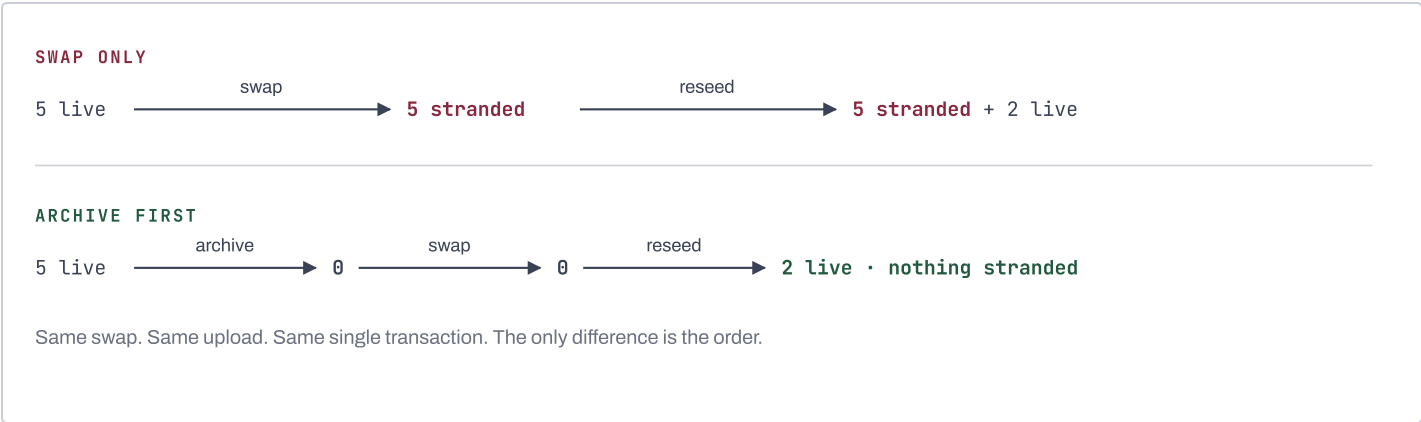
UNRESOLVED

the one capability we need from Canton — see below

Why the order matters

Archiving has to happen before the swap, and this is why.

A **package** is code; it is *vetted* or not. A **contract** is data; it is *active* or *archived*. The swap edits the allowlist of code that may run — it does not migrate, convert or remove any contract. So old contracts are simply left where they are, and for an incompatible change nothing can read them any more.



Archiving must precede the swap. Afterwards the old package is unvetted, so its contracts can no longer be exercised — including the archive choice itself.

The two options

One decision rule: is the change a valid upgrade?

CHANGE	OPTION	DEPENDENT PACKAGES	STATUS
Compatible add optional field, add a choice	bump 1.0.0 → 1.0.1	resolve automatically — no rebuild	SHIPPED
Incompatible change a type, drop a field, change a signatory	archive → swap → reseed	must be rebuilt too	WORKS NEEDS A HOME

Issues with each

Option 1 — bump the version

- Accepted live while the old version holds contracts; both versions vetted at once
- Dependents keep working **unrebuilt** — Canton resolves templates by package name and selects the newest compatible version
- **Limit:** incompatible changes are rejected — `NOT_VALID_UPGRADE_PACKAGE`
- **Forcing past that is a dead end** (below)

Option 2 — archive and replace

- Requires holding every party's signatory authority to archive — true in a dev sandbox, which is what scopes this to development
- **Order is load-bearing:** archive before the swap or the contracts become unremovable
- If other packages depend on the one you change, **they must be rebuilt too**
- Leaves history in the event log

FORCING AN INCOMPATIBLE VERSION BUMP

REJECTED AS A DESIGN

```
force-vet mirrors 1.0.1 (Int) alongside 1.0.0 (Text)
FORCE_VET_INCOMPATIBLE_BUMP ⇒ SUCCEEDED
  mirrors 1.0.1  510abec1a117  VETTED
  mirrors 1.0.0  c17b5cb16a9d  VETTED

then attempt to use either shape
create  COMMAND_PREPROCESSING_FAILED: mismatching type: Int64 and value: Text(...)
archive INTERPRETATION_UPGRADE_ERROR_TRANSLATION_FAILED: Translation...
```

Canton permits the force. The result is a package name usable for neither shape, with contracts that can no longer be archived — strictly worse than not acting.

PACKAGE B DEPENDS ON A • A REPLACED • B NOT REBUILT

WHY DEPENDENTS MUST BE REBUILT

```
SWAP items oldA=f598c7e1de1b  newA=04d8ea17faef
PEEK holders=1                ← B still SEES its contract
FAILURE: PACKAGE_SELECTION_FAILED — No synchronizers satisfy
      the topology requirements    ← but cannot use it
```

A package's compiled code embeds the package IDs of everything it depends on. So when A changes, B is still compiled against the old A — and the old A is no longer vetted.

What we decided

Scope for the proposal.

Option B is the approach. Archive the old contracts, then replace the package under the same name and version, on a running participant.

- **Propose nothing for compatible changes.** Bumping the version is the supported path and already works live.
- **We chose option B — archive, then replace —** for incompatible changes, with archiving strictly before the swap.
- **Drop the protocol-event request.** Real archive events are emitted already, so consumers converge unaided.
- **Build the workflow in DPM.** Canton needs no change for the loop itself.
- **Ask Canton for one capability —** reclaiming storage on an in-memory participant.

What works on a database, and what is missing on the sandbox

Canton already has the cleanup mechanisms. Availability is the problem.

MECHANISM	IN-MEMORY SANDBOX	POSTGRES-BACKED
<code>repair.purge</code> deletes active contracts outright, no archive events	REFUSED "is in memory which is not supported by repair. Use db persistence."	VERIFIED 2 active → 0 requires disconnect first
pruning deletes archived contracts and history	REFUSED	NO SUCCESSFUL RUN <code>UNSAFE_TO_PRUNE</code> 168h deduplication window
<code>dars.remove</code> deletes the package itself	WORKS once unvetted and unreferenced	WORKS

Pruning failed identically on both backends — a time and safety constraint rather than a storage one — so a database does not resolve it. `repair.purge` is the mechanism whose only blocker is storage type.

A Postgres-backed sandbox is also not a workaround in practice: `dpm sandbox` re-runs topology initialisation on startup and fails against already-initialised databases, so it is not restart-safe.

The proposed fix in DPM

Three existing calls, driven by a config file.

THE RELOAD

NO FORCE FLAG, NO REPAIR TOOLING, NO NEW API

```
// 1. archive the old contracts – ordinary DamL Archive, as the signatories.
//   The only valid window: after the swap the old package is unvetted and
//   nothing from it can be exercised, archiving included.

// 2. upload the rebuilt package, unvetted
val newId = sandbox.dars.upload(dar, vetAllPackages = false)

// 3. swap the allowlist in ONE topology transaction: old out, new in
sandbox.topology.vetted_packages.propose_delta(
  participant = sandbox.id,
  adds        = Seq(VettedPackage(newId, None, None)),
  removes     = Seq(oldId),
  store       = TopologyStoreId.Synchronizer(psid),    // not the default
)

// 4. reseed           5. dars.remove(oldId)
```

Two details that matter. `vetAllPackages = false` is what allows an upgrade-incompatible build to upload at all. And `store` must name the synchronizer: `propose_delta` defaults to the Authorized store, which is empty on a fresh participant, so a defaulted call emits serial 1, changes nothing, and returns success.

If more than one package is involved

Only relevant when packages depend on each other — if your project is a single package, the five steps above are the whole loop. Where **B depends on A** and A changes, B has to be rebuilt and swapped in the same operation:

```
1. rebuild A
2. rebuild B against the new A           dpm already builds in dependency order
3. archive B's contracts, then A's       B first – B holds references into A
4. upload new A and new B, both unvetted
5. ONE topology transaction:
    adds    = [ newA, newB ]
    removes = [ oldA, oldB ]
6. reseed
```

Both packages go on the allowlist in the same transaction, so B is never vetted while the A it needs is missing.

Configuration

Every destructive step is a switch, because not every project wants every step.

The loop performs several independent destructive actions. Some teams will want contracts archived but the old package kept; some will want history reclaimed and some will not; some will want to inspect stranded contracts deliberately. So the workflow is driven by a declarative file rather than hard-coded behaviour.

DPM-DEV.YAML

PROPOSED

```
apiVersion: digitalasset.com/v1
kind: DevReload
spec:
  watch: [ mirrors/daml, holders/daml ]
  packages: [ mirrors, holders ]
  rebuildDependents: true  # if B uses A, rebuild B when A changes

  onReload:
    archiveContracts: true  # clear the ACS before swapping.
                           # false → faster, but strands contracts
    removeOldPackage: true  # dars.remove after the swap
    reseed:
      enabled: true
      script: Seed:seed
    reclaimHistory: off    # off | purge | prune
                           # requires Canton support – see below

  onStranded: warn        # warn | fail | ignore
  requireDevParticipant: true
```

Each key maps one-to-one onto a step that has been verified independently, so any combination is a real configuration rather than an untested path.

KEY	CONTROLS	IF DISABLED
archiveContracts	whether the ACS is cleared before the swap	contracts are stranded — <code>dars.remove</code> then also becomes impossible
removeOldPackage	<code>dars.remove</code> after the swap	old package stays on the participant, unvetted and inert
reseed	re-running the seed script, and which one	ledger left empty after the reload
reclaimHistory	discarding archived contracts and event-log history	history accumulates until the participant restarts
onStranded	reaction when stranded contracts are detected	<code>fail</code> makes a mis-ordered reload an error instead of a warning

Capability, not just preference. `reclaimHistory` cannot be honoured on an in-memory participant today. The tool should report that plainly rather than fail or silently ignore it:

```
dpm dev: reclaimHistory=purge requested, unavailable on this participant
(storage=memory; repair operations require db persistence)
continuing without history reclamation
```

Configuration declares intent; the tool reports what it was actually able to do.

What Canton has to provide

Make `repair.purge`, or an equivalent, available on an in-memory development participant. With it, contracts are deleted directly rather than archived, so no history accumulates and `reclaimHistory` becomes honourable. One existing feature, one storage restriction lifted.

Toolchain defects, to file separately

- `dpm sandbox` cannot execute any Daml transaction on an Oracle JDK — the fat jar bundles BouncyCastle unsigned and Oracle enforces JCE provider signing. Upload and party allocation succeed, so the environment appears healthy until the first submit fails as an opaque `INTERNAL`.
- `dpm new --template empty-skeleton` omits `sdk-version`, after which `dpm` exposes no subcommands at all.